

모두를 위한 딥러닝 (입문편)

허 찬

강의 계획표 - 모두를 위한 딥러닝 (입문편)

학습주제(강좌명)	차시(모듈)	차시별 주제	학습내용
모두를 위한 딥러닝 (입문편)	1	Dive into deep learning	머신러닝, 딥러닝, 인공신경망의 개념 이해 딥러닝의 분류 및 응용 분야 이해 딥러닝 모델의 장/단점 이해
	2	딥러닝 코딩 환경 준비	머신러닝/딥러닝 워크플로우의 이해 파이썬 기반 머신러닝, 딥러닝 프레임워크의 이해 패키지 설치 및 기초 벡터 연산 설명
	3	딥러닝 모델 학습의 4steps	딥러닝 학습 과정의 단계별 이해
	4	데이터 시각화	데이터 시각화 Pandas, Matplotlib, Seaborn 패키지의 사용
	5	딥러닝을 이용한 분류 모델 실습	딥러닝 코드 실습을 통한 분류 문제 해결
	6	딥러닝을 이용한 회귀 모델 실습	딥러닝 코드 실습을 통한 회귀 문제 해결
	7	딥러닝 모델의 성능 향상 기법	딥러닝 기본 모델의 성능 향상 기법

1.7	딥러닝 모델의 성능 향상 기법	다양한 딥러닝 모델의 성능 향상 기법
-----	------------------	----------------------

Contents

- 과적합의 이해
- 딥러닝 모델에 영향을 주는 다양한 요소 이해
- 딥러닝 모델의 다양한 성능 향상 기법

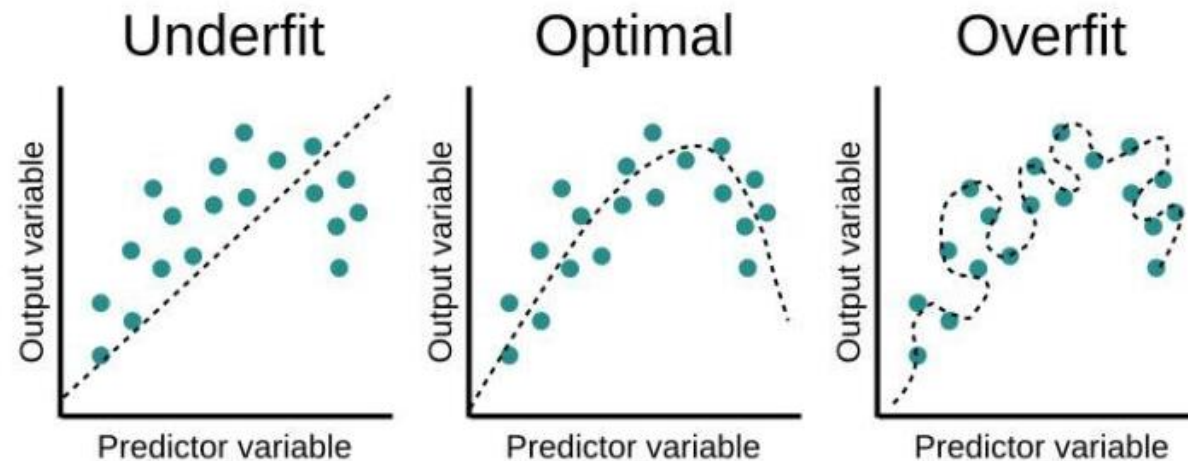
1.7 과적합의 이해

- 과대적합 (Overfitting)

학습 데이터에 대해서만 너무 학습이 잘 되어서 테스트 데이터에 대한 성능이 떨어지는 단계

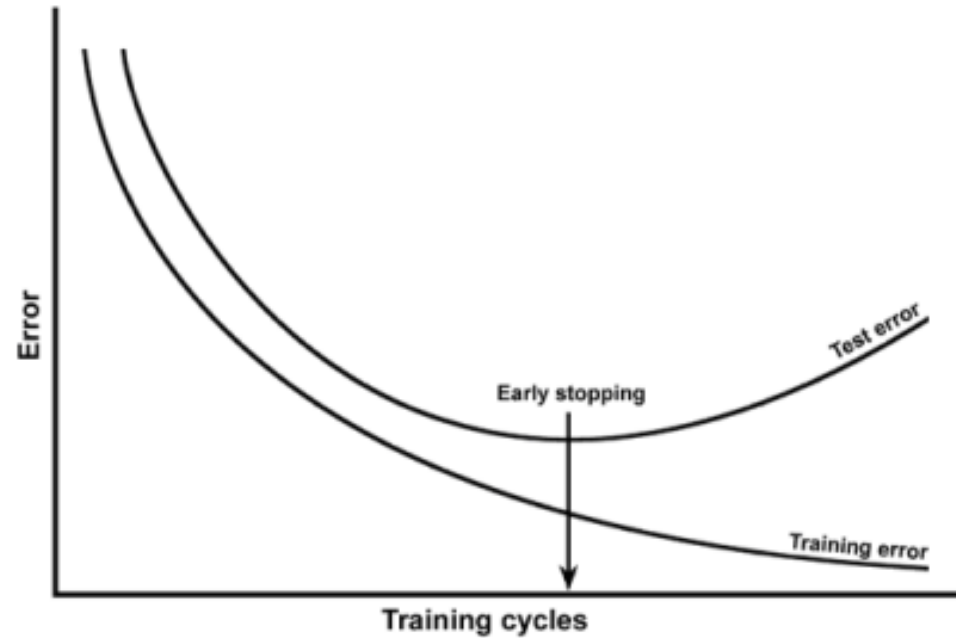
- 과소적합 (Underfitting)

학습이 덜 이루어지는 단계



1.7 과적합의 이해

과적합의 확인



- Train loss는 계속 감소하는데 Test set의 loss가 증가하는 경우
- 혹은 Train accuracy는 향상하는데 Test accuracy가 감소하는 경우

1.7 과적합의 이해

과적합의 방지 방법

0. Early stopping 적용

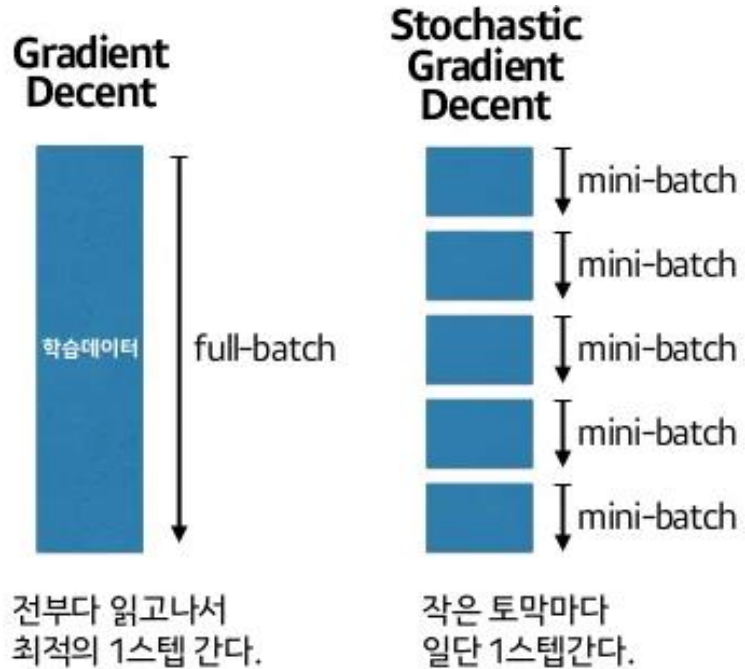
1. 데이터 양 늘리기 (Data augmentation)

2. 모델 복잡도 줄이기

3. 가중치 규제 적용

4. Dropout, normalization 등의 방법 사용

1.7 딥러닝의 요소 : Batch size



(Full) Batch training

- 학습을 통해 한번 모델이 업데이트 될 때 데이터 전체를 가지고 모델의 가중치를 업데이트하는 과정

Mini-batch training

- 학습을 통해 한번 모델이 업데이트 될 때 데이터 일부를 가지고 모델의 가중치를 업데이트하는 과정

1.7 딥러닝의 요소 : Batch size

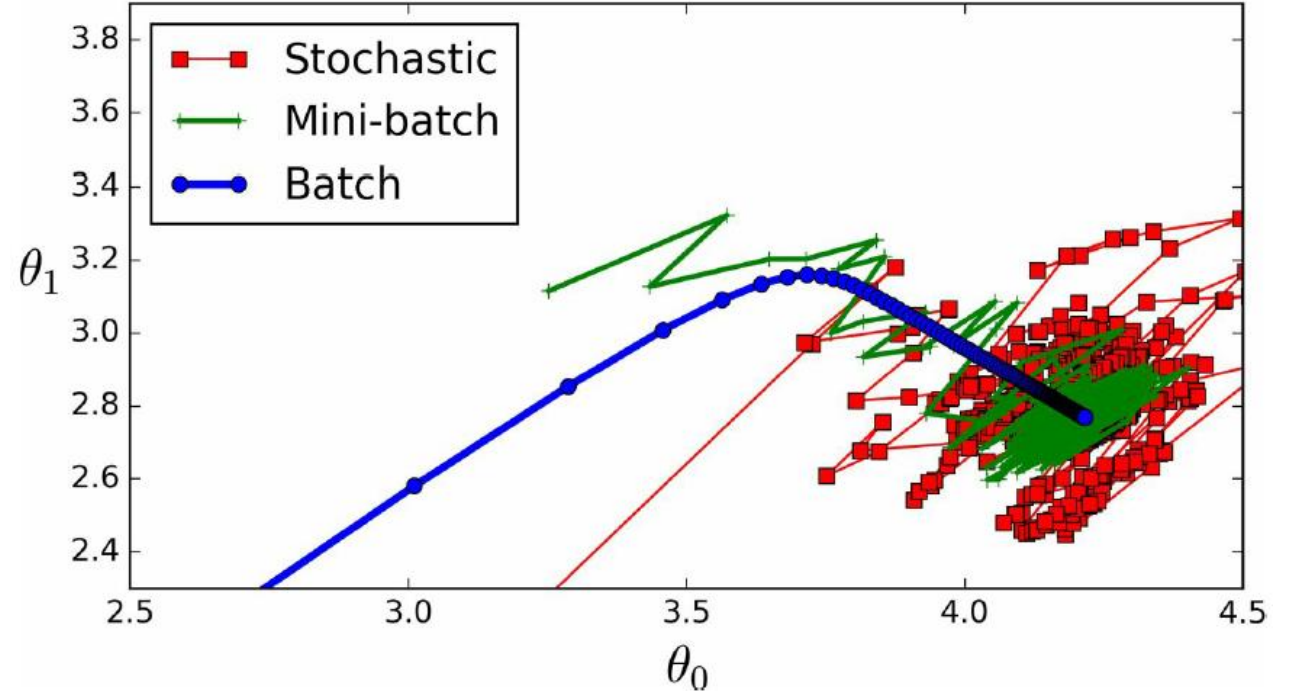
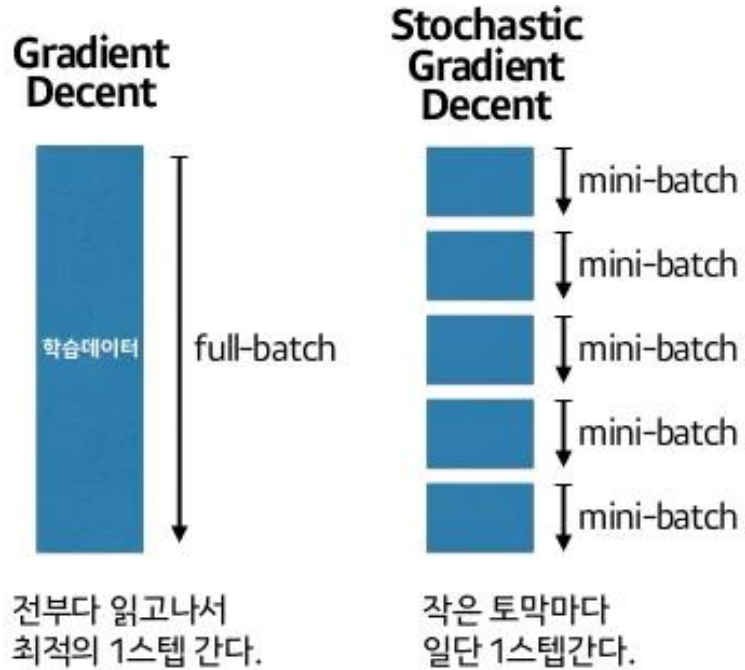
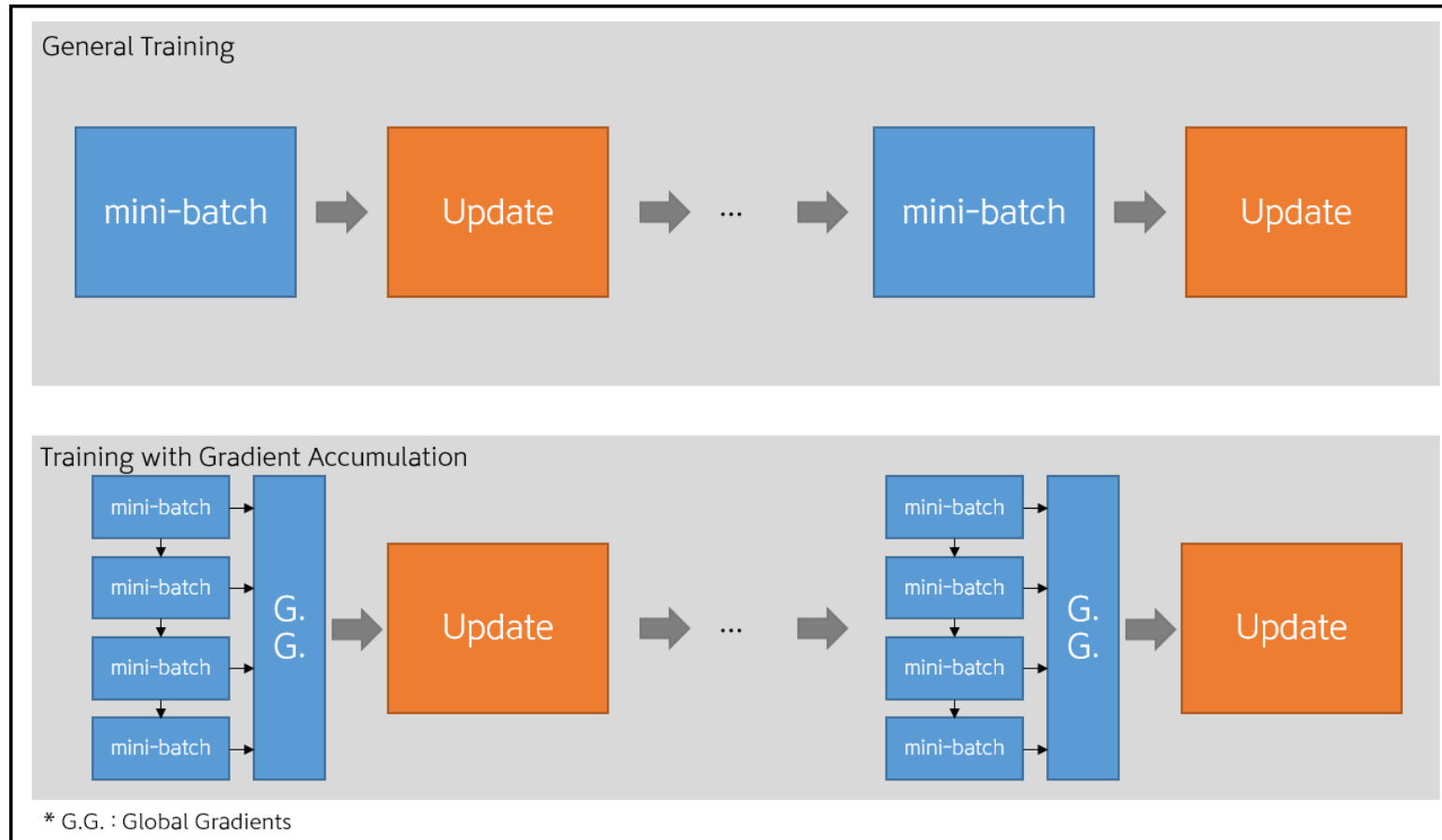


Figure 4-11. Gradient Descent paths in parameter space

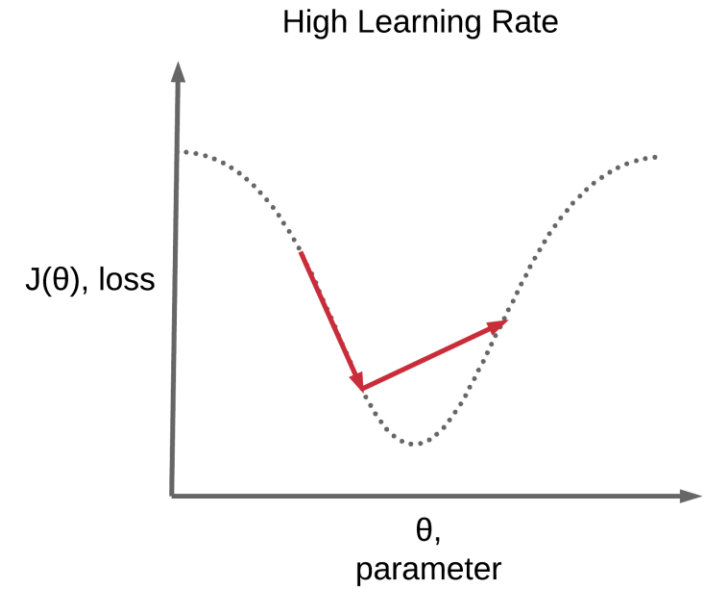
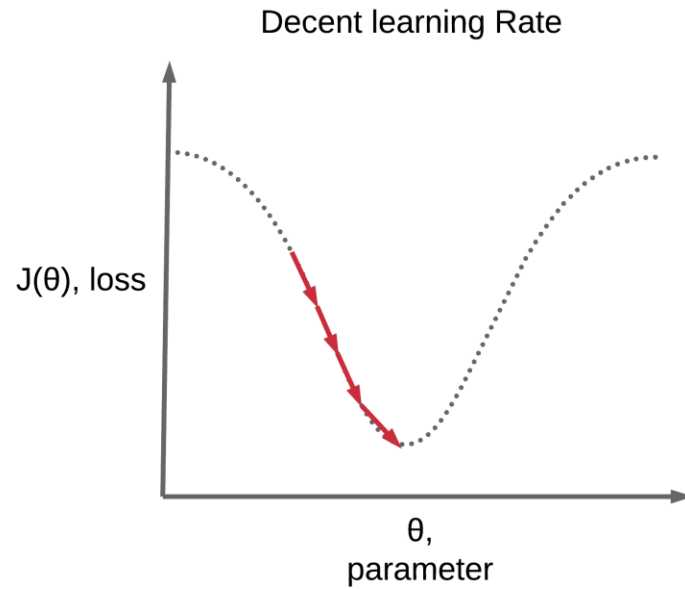
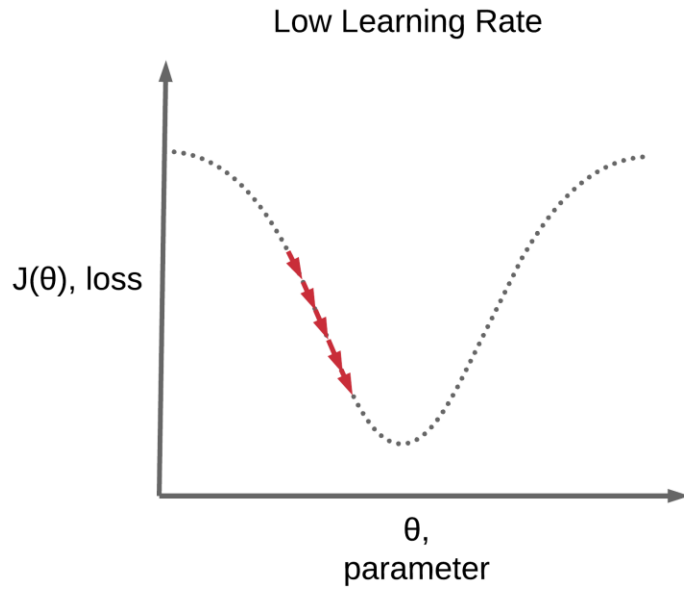
Full batch vs Mini-batch vs one-data

1.7 딥러닝의 요소 : Batch size



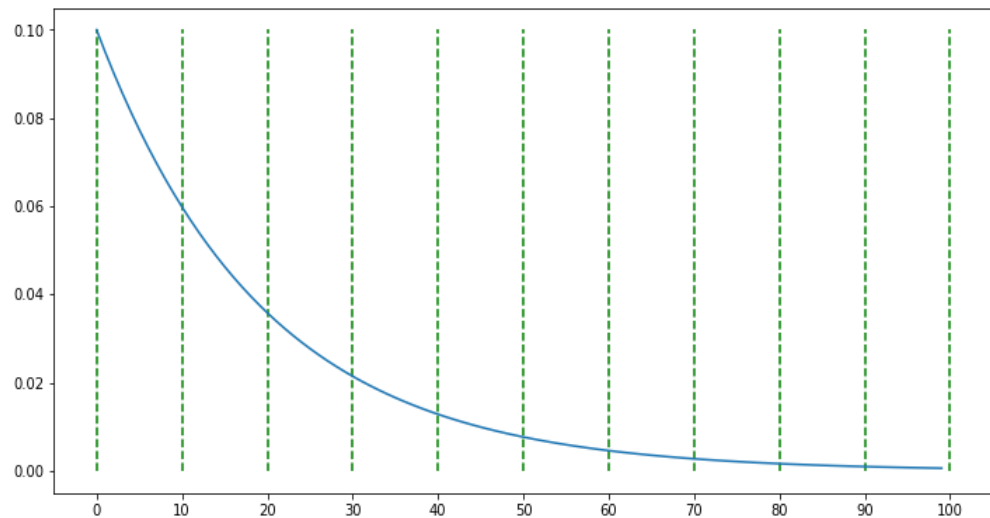
- Gradient accumulation을 이용한 batch size 제한 문제 해결

1.7 딥러닝의 요소 : Learning rate

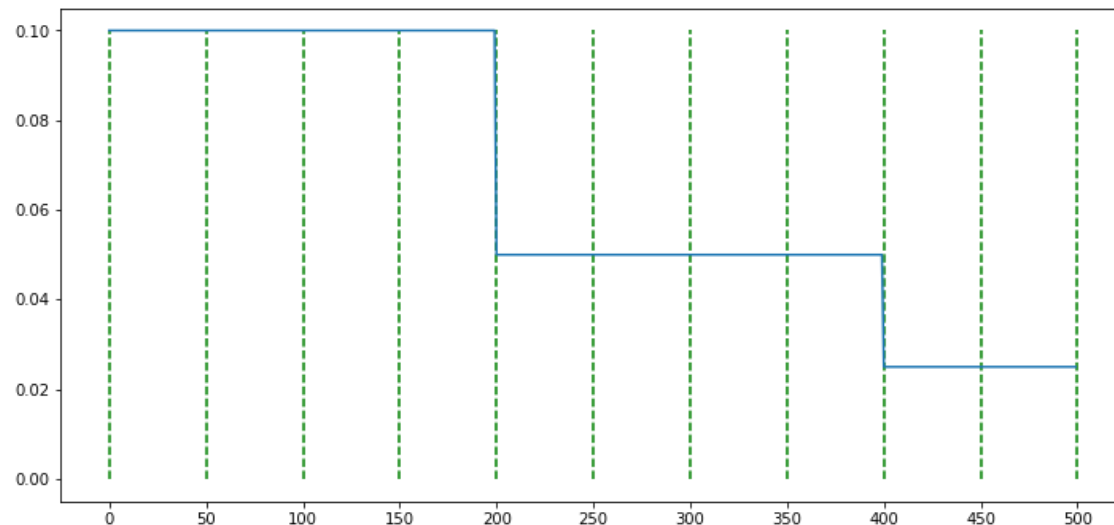


학습률을 어떻게 조정하는게 좋을까?

1.7 딥러닝의 요소 : Learning rate

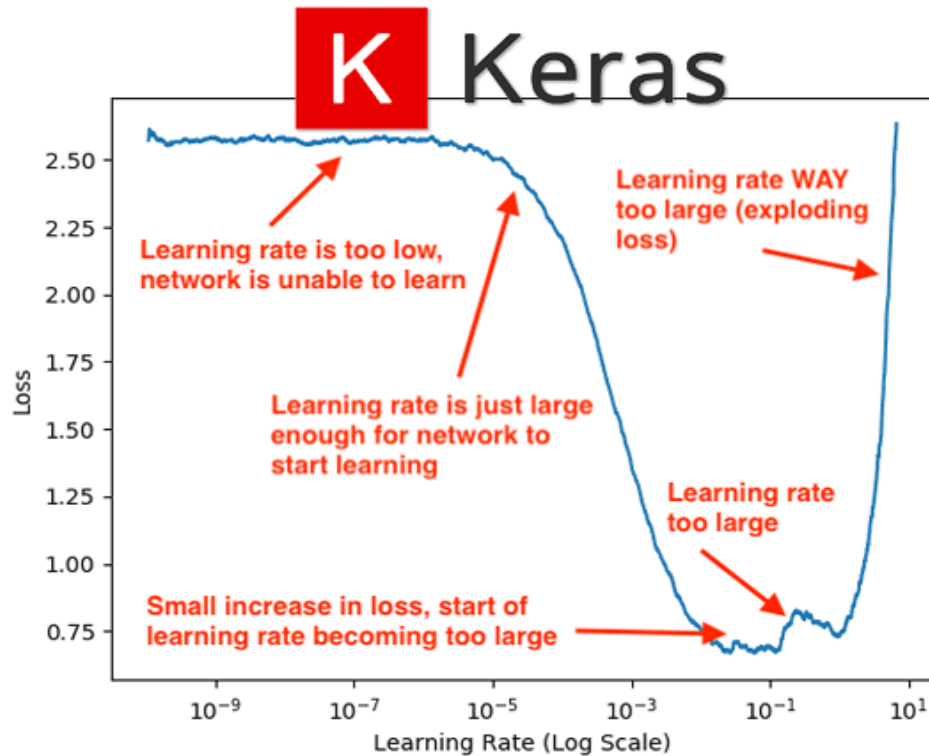


Lambda LR scheduler



Step LR scheduler

1.7 딥러닝의 요소 : Learning rate



Learning rate scheduler:

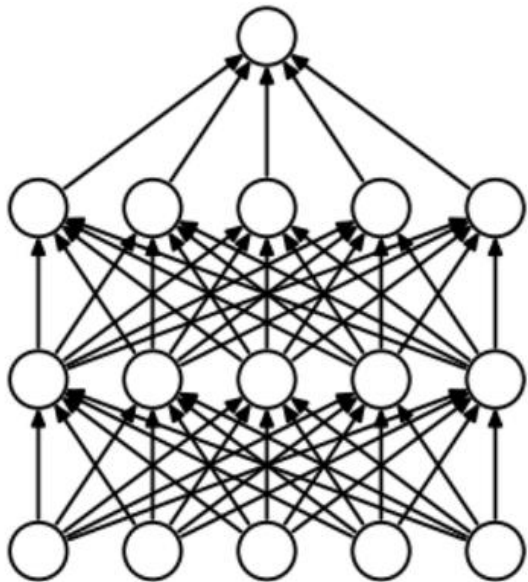
- 학습시 속도를 초반에는 빠르게, 학습이 어느 정도 진행된 이후에는 느리게 만들어주는 tool.
- `keras.callbacks.LearningRateScheduler(schedule, verbose=0)`

1.7 딥러닝의 성능 향상 방법들

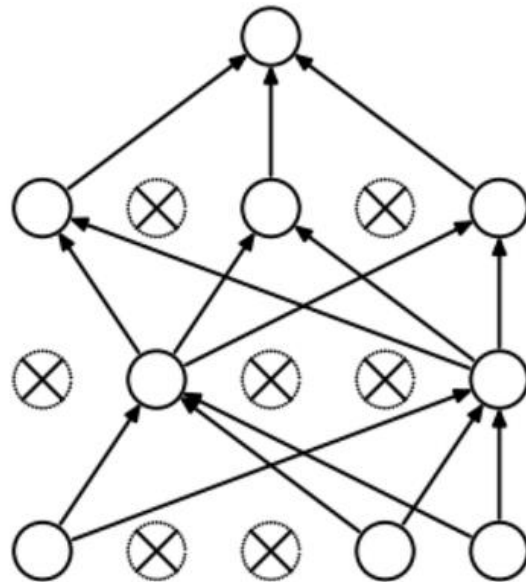
1. Dropout
2. Batch normalization
3. Regularization

1.7 딥러닝의 성능 향상 방법들

1. Dropout



(a) Standard Neural Net



(b) After applying dropout.

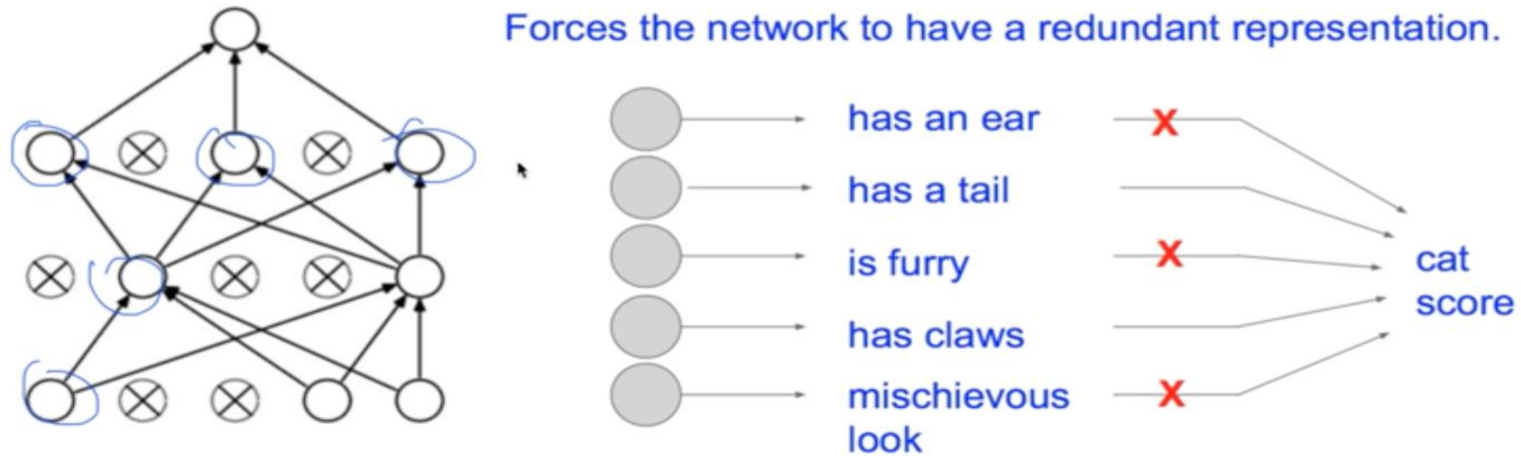
- Dropout은 신경망의 뉴런을 부분적으로 생략하여 모델의 과적합(overfitting)을 해결해 주기 위한 방법.
- 과적합이란? >> 모델이 학습 데이터만 정답을 잘 맞추고 테스트 데이터에서 성능이 떨어지는 현상
- 학습할때 계속 연결되는 node를 바꿈으로써 특정 weight에 집중되는 현상을 막음.

1.7 딥러닝의 성능 향상 방법들

1. Dropout

Waaaaait a second...

How could this possibly be a good idea?

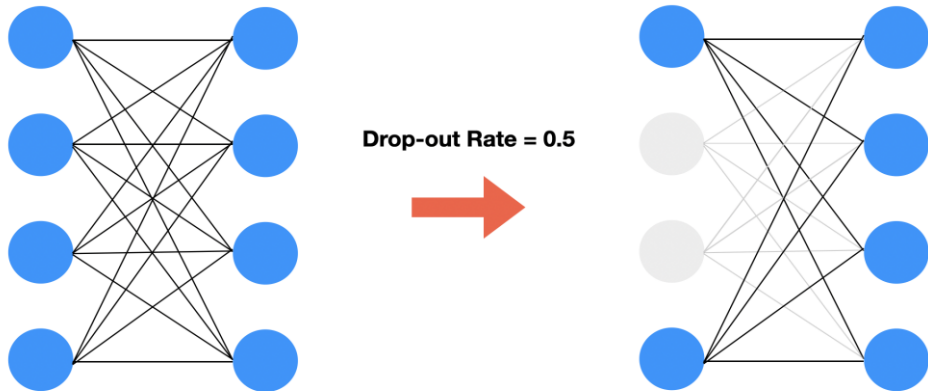


Fei-Fei Li & Andrej Karpathy & Justin Johnson

Lecture 6 - 53

25 Jan 2016

1.7 딥러닝의 성능 향상 방법들



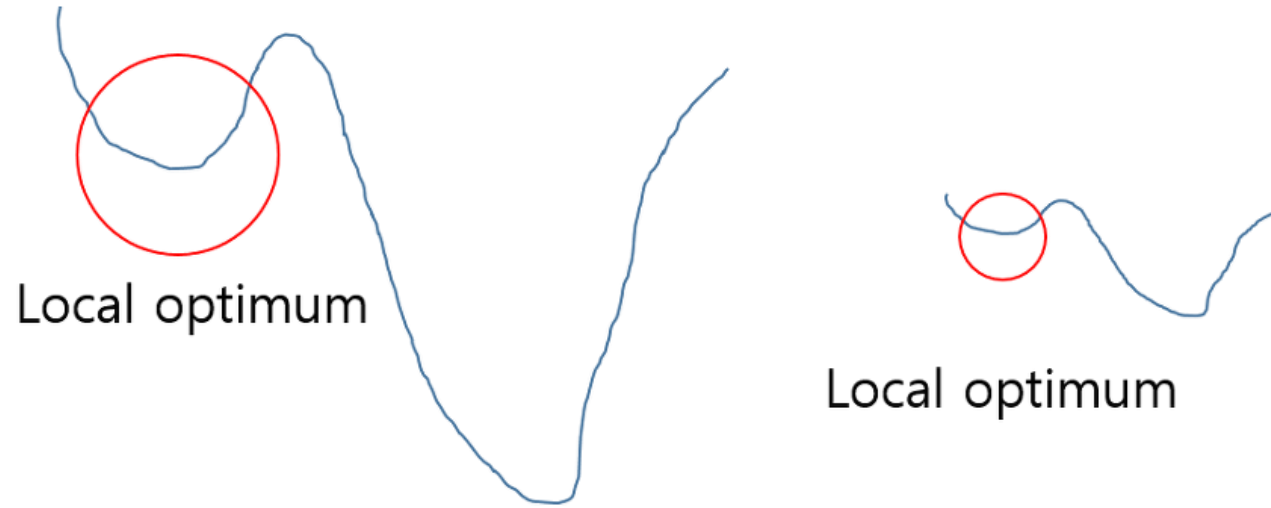
<https://heytech.tistory.com/>

Dropout layer:

- Drop-out은 어떤 특정한 Feature만을 과도하게 집중하여 학습함으로써 발생할 수 있는 과대적합(Overfitting)을 방지하기 위해 사용.
- `model.add(tf.keras.layers.Dropout(drop_rate))`

1.7 딥러닝의 성능 향상 방법들

2. Batch normalization



- (좌) Normalization 적용 전 / (우) Normalization 적용 후

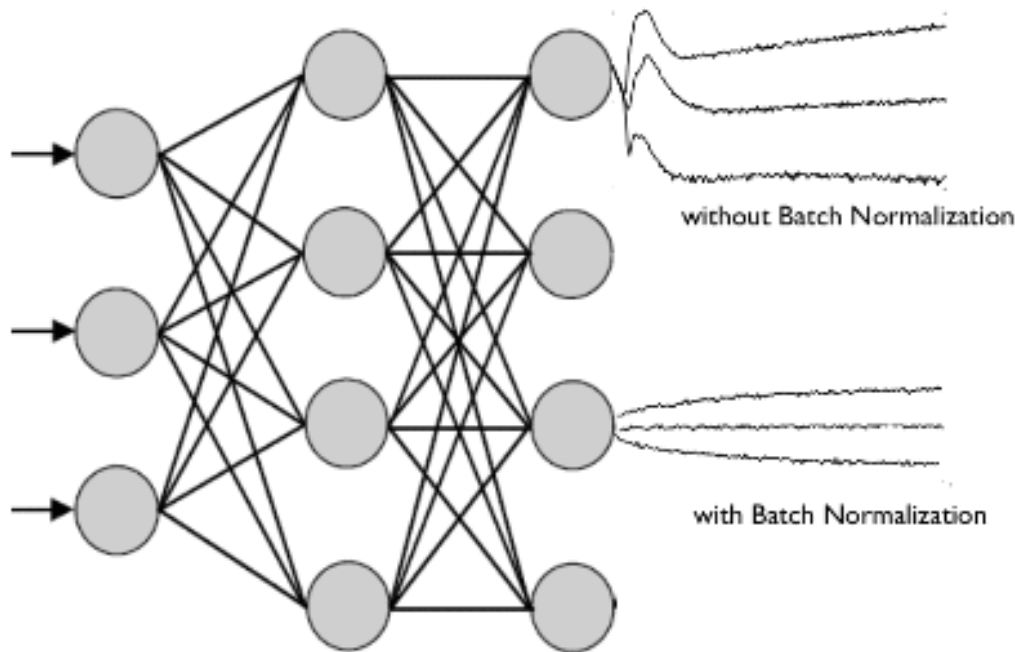
정규화를 하는 이유:

1) 빠른 학습

2) Local optimum 문제에 빠지는 가능성을 줄임.

1.7 딥러닝의 성능 향상 방법들

2. Batch normalization



- 배치 정규화 논문에서는 학습에서 불안정화가 일어나는 이유를 'Internal Covariance Shift' 라고 주장. 이는 네트워크의 각 레이어나 Activation 마다 입력값의 분산이 달라지는 현상을 뜻함.
- Covariate Shift : 이전 레이어의 파라미터 변화로 인하여 현재 레이어의 입력의 분포가 바뀌는 현상.
- Internal Covariate Shift : 레이어를 통과할 때 마다 Covariate Shift 가 일어나면서 입력의 분포가 약간씩 변하는 현상.

<https://eehoeskrap.tistory.com/430>

1.7 딥러닝의 성능 향상 방법들

3. Regularization

L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$

L1 정규화

- 편미분 을 하면 w값은 상수값이 되어버리고, 그 부호에 따라 +-가 결정
- 가중치가 너무 작은 경우는 상수 값에 의해서 weight가 0이 되어버림.

L2 정규화

- Loss function에 제공한 가중치 값을 더해줌으로써 편미분을 통한 backpropagation시 loss 뿐만 아니라 가중치 또한 줄어드는 방식으로 학습

1.7 딥러닝의 성능 향상 방법들

3. Regularization

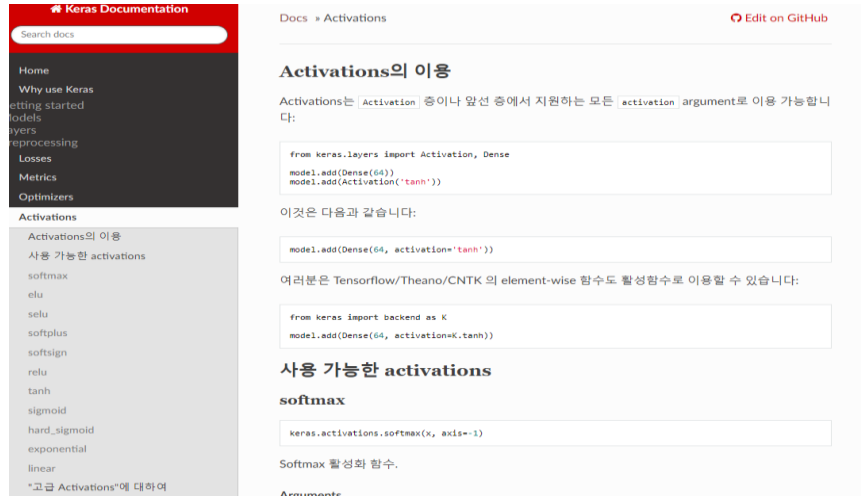
L1 정규화

- L1은 통상적으로 상수 값을 빼주도록 되어 있기 때문에 특정 weight들은 0으로 수렴.
- 그러므로 몇 개의 의미 있는 값을 고집어 내고 싶은 경우에는 L1이 효과적이기 때문에 sparse model에 적합하다.
- 단 미분이 불가능한 점이 있기 때문에 gradient-based learning에 적용시에는 주의가 필요.

L2 정규화

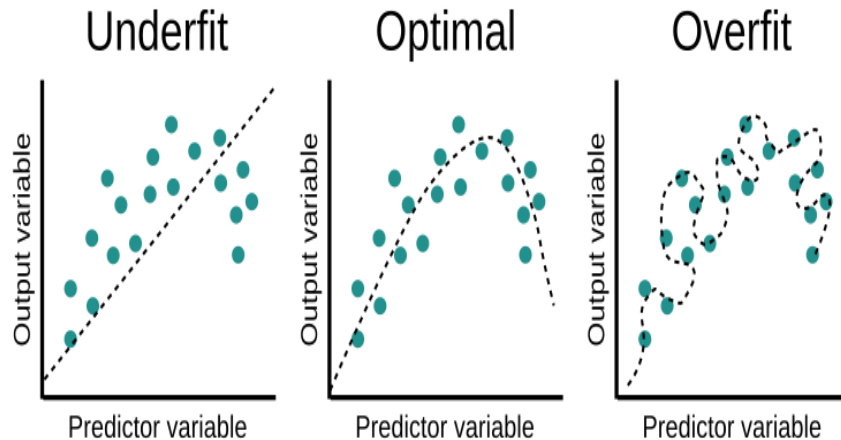
- L2 : L2는 outlier에 대해선 L1보다 덜 Robust(둔감).
- 따라서 Outlier에 대해 신경을 써야하는 경우에 사용.

1.7 딥러닝의 성능 향상 방법들



Activation function:

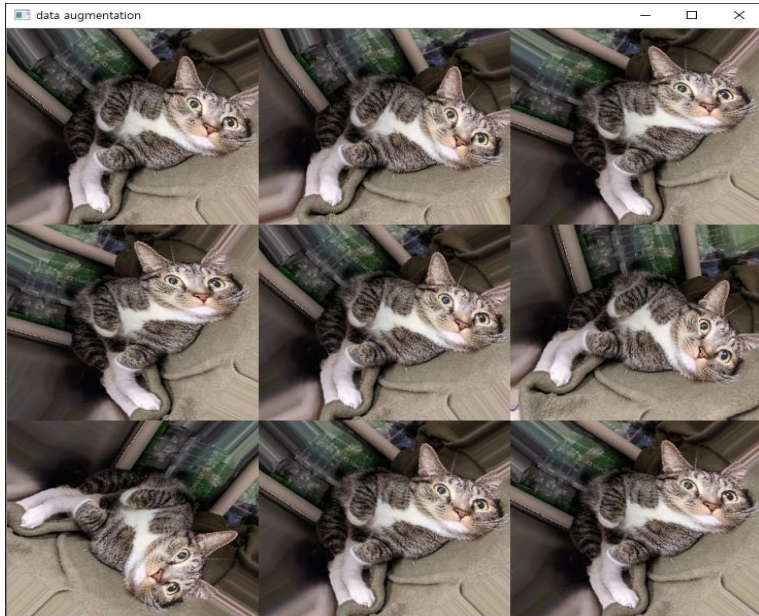
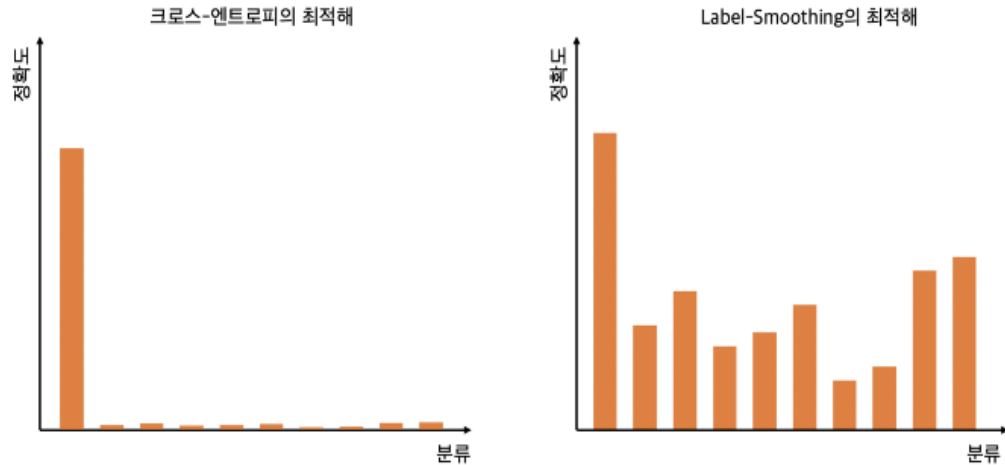
- GeLu, Leaky relu 등 다양한 발전 모델
- Keras activation



Weight decay:

- weight 값을 일정 범위 안에 고정시켜 과적합 방지.
- `layer = layers.Dense(...
kernel_regularizer=regularizers.L1L2(l1=1e-5, l2=1e-4),
bias_regularizer=regularizers.L2(1e-4),
activity_regularizer=regularizers.L2(1e-5))`

1.7 딥러닝의 성능 향상 방법들



Label smoothing:

- 과적합(Overfitting) 방지: 딥러닝 모델이 훈련 데이터에 지나치게 적응되어 새로운 데이터에 대한 일반화 능력이 저하되는 것을 방지. 훈련 데이터에 있는 라벨에 대해 완벽하게 맞추려고 하는 모델은 높은 복잡성을 가질 수 있으며, 이는 새로운 데이터에 대한 예측을 어렵게 만듭니다.
- Soft Decision Boundary: 라벨 스무딩은 모델이 클래스 예측을 더 부드럽게 만들어주어, 모델의 결정 경계가 더 매끄럽고 일반적인 패턴을 학습하게 합니다. 이는 더 좋은 일반화 능력을 가져올 수 있습니다.
- `tf.keras.losses.CategoricalCrossentropy(from_logits=False, label_smoothing=0.0...)`

Data augmentation:

- 이미지 데이터 augmentation을 통해 학습 데이터를 늘려 주는 방법
- `keras.preprocessing.image.ImageDataGenerator(featurewise_center=False, ...width_shift_range=0.0, height_shift_range=0.0, brightness_range=None, shear_range=0.0, zoom_range=0.0, ..., cval=0.0, horizontal_flip=False, vertical_flip=False, rescale=None, preprocessing_function=None, data_format=None, validation_split=0.0, dtype=None)`

감사합니다